

FIFO DEPTH CALCULATION

VLSI INTERVIEW PREPARATION



TABLE OF CONTENTS

- **INTRODUCTION**

- Key Characteristics of FIFO
- where is FIFO used ?
- Why is FIFO Depth Important

- **UNDERSTANDING FIFO DEPTH**

- Key aspects of FIFO Depth
- Factors affecting FIFO Depth

- **FIFO DEPTH CALCULATION**

- Practical cases & Scenarios for FIFO Depth Calculation
- Case – 1 : $f_A > f_B$ with no idle cycles in both write and read.
- Case – 2 : $f_A > f_B$ with one clk cycle delay between two successive reads and writes.
- Case – 3 : $f_A > f_B$ with idle cycles in both write and read.
- Case - 4 : $f_A > f_B$ with duty cycles given for wr_enb and rd_enb.

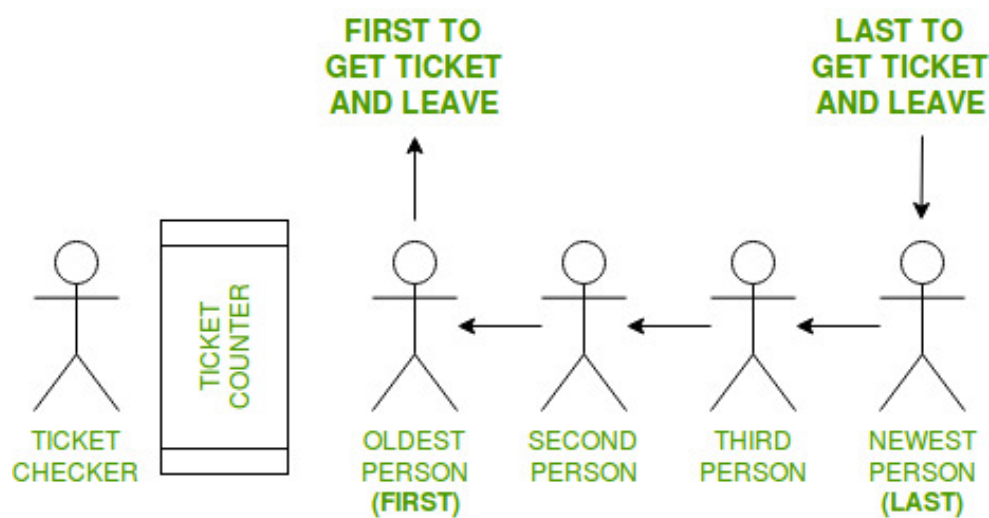
- Case — 5 : $f_A < f_B$ with no idle cycles in both write and read (i.e., the delay between two consecutive writes and reads is one clock cycle).
- Case — 6 : $f_A = f_B$ with no idle cycles in both write and read (i.e., the delay between two consecutive writes and reads is one clock cycle).

- **COMMON MISTAKES & OPTIMIZATION TIPS**

- Avoiding Underflow & Overflow
- Choosing an Optimal FIFO Depth

INTRODUCTION

FIFO (First-In-First-Out) is a data buffering mechanism used in digital systems where the first data that enters the buffer is the first data to be processed or removed. It works like a queue in real life, similar to waiting in line at a store whoever arrives first is served first.



KEY CHARACTERISTICS OF FIFO :

- **Order Preservation:** Data is read in the same order it was written.
- **No Overwriting:** Once full, new data cannot be written unless old data is read out.
- **Used for Data Synchronization:** Helps in handling clock domain crossing, buffering, and rate mismatches between different components.

WHERE IS FIFO USED ?

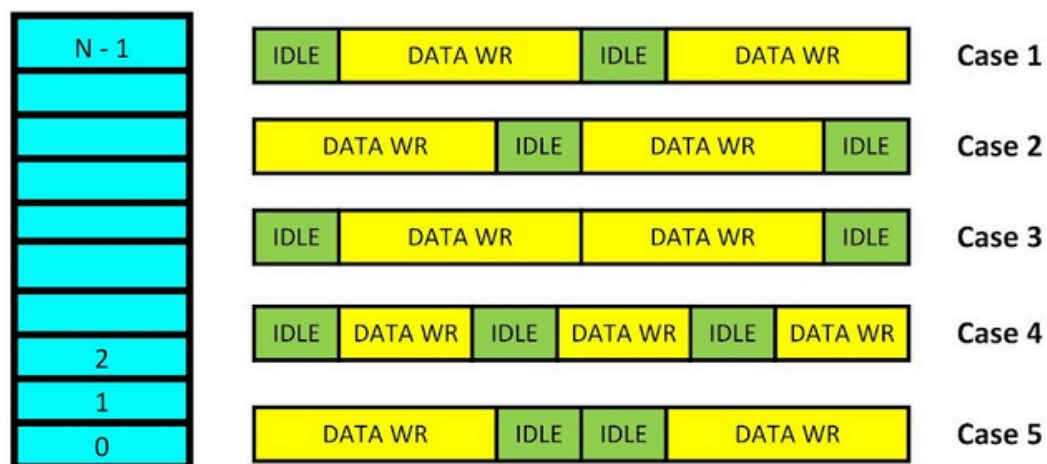
- **Processor and Memory Communication** – Buffers data between CPU and RAM.
- **Network Buffers** – Ensures smooth data flow in routers and switches.
- **Audio & Video Streaming** – Handles real-time data transmission.
- **Clock Domain Crossing (CDC)** – Transfers data between circuits operating at different clock speeds.

WHY IS FIFO DEPTH IMPORTANT ?

- **Prevents Data Loss (Overflow)**
 - If the FIFO is too small, incoming data may not have space, causing data loss or corruption (FIFO overflow).
- **Prevents Stalling (Underflow)**
 - If the FIFO is too large, it might lead to unnecessary memory usage and longer delays in processing data (FIFO underflow).
- **Ensures Smooth Data Transfer**
 - Helps in bridging data rate mismatches between fast and slow components.
- **Optimizes Hardware Resources**
 - Using the right FIFO depth saves FPGA memory, power, and computation time.

UNDERSTANDING FIFO DEPTH

FIFO depth refers to the number of storage locations (memory entries) available in a FIFO buffer. It determines how much data the FIFO can temporarily hold before it is read and processed.



KEY ASPECTS OF FIFO DEPTH :

- Measured in words or bytes depending on data size.
- Directly affects latency, throughput, and performance in digital systems.
- Should be optimized to avoid data loss (overflow) or starvation (underflow).

$$\text{FIFO Depth} = \frac{\text{Max Data Rate} \times \text{Latency}}{\text{Min Data Rate}}$$

This formula helps determine the required FIFO size based on data flow and processing speed.

FACTORS AFFECTING FIFO DEPTH

1. DATA RATE (WRITE VS. READ SPEED)

- If the write speed is higher than the read speed, the FIFO needs more depth to hold excess data.
- If both speeds are equal, FIFO depth can be minimal.

2. CLOCK DOMAIN DIFFERENCES

- In asynchronous FIFO designs (different clock domains), extra depth is required to compensate for clock variations.
- Example: Transferring data from a 100 MHz domain to a 50 MHz domain requires additional FIFO depth.

3. BURST DATA TRAFFIC

- Some systems handle bursts of data, requiring temporary storage before processing.
- Example: USB data transfer often sends data in bursts, requiring FIFO buffers to handle sudden spikes.

4. PROCESSING LATENCY

- FIFO depth must account for latency caused by delays in processing or decision-making logic.
- Example: A digital signal processor (DSP) pipeline may introduce latency, requiring a deeper FIFO.

5. MEMORY CONSTRAINTS

- In FPGA or ASIC designs, FIFO depth should be optimized to balance performance and memory usage.
- Example: A high-depth FIFO might be impractical for low-power embedded devices.

6. ERROR HANDLING & RETRANSMISSION

- In some systems, FIFO depth is increased to handle retransmissions in case of errors.
- Example: TCP/IP packet buffering ensures reliable data delivery despite network fluctuations.

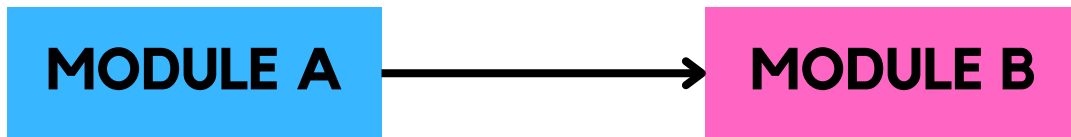
FIFO DEPTH CALCULATION

An asynchronous FIFO is essential when transferring data between two modules operating in different clock domains to prevent data loss. It is particularly required when the writing speed is faster than the reading speed, as it buffers the unread data until the slower module can process it.

The FIFO depth should be designed to accommodate all the data that remains unread by the slower module. However, FIFO buffering is effective only when data is transmitted in bursts rather than a continuous stream. If data flows continuously without interruption, the FIFO size would theoretically need to be infinite. To determine the appropriate FIFO size, factors such as burst rate, burst size, and operating frequencies must be considered.

The approach to determining the FIFO size is based on calculating the number of data items that remain unread during the writing process. In other words, the FIFO depth should be equal to the number of data items left unprocessed. Using this logic, I have estimated the required FIFO size for various scenarios without relying on standard formulas, as they may not be applicable in every case.

The following examples illustrate various scenarios where an asynchronous FIFO is necessary. In these cases, module 'A' intends to transmit data to module 'B'.



PRACTICAL CASES & SCENARIOS FOR FIFO DEPTH CALCULATIONS

FIFO depth calculations play a crucial role in ensuring smooth data transfer between systems operating at different clock speeds. In high-speed communication systems where the write clock (WR_CLK) is higher than the read clock (RD_CLK), data accumulates faster than it is consumed. For example, if data is written at 100 MHz and read at 80 MHz with a burst length of 256 words, the FIFO must be deep enough to handle the rate mismatch. A safety margin of 25–30% is often added to prevent overflow. In contrast, video processing applications where RD_CLK is greater than WR_CLK require a smaller FIFO since data is consumed more quickly than it is written. For instance, a camera generating frames at 30 FPS while a processor reads at 60 FPS requires a FIFO depth that ensures frames are available without causing underflow.

Multi-clock domain systems introduce additional challenges due to phase differences between WR_CLK and RD_CLK. For example, transferring data from a 100 MHz domain to a 50 MHz domain requires considering synchronization delays and metastability issues. In such cases, dual-clock FIFOs with appropriate depth calculations help ensure reliable data transfer. Similarly, networking applications require FIFO depth optimization to handle peak loads. For instance, an Ethernet module transmitting at 1 Gbps while the CPU processes at 500 Mbps must have a FIFO deep enough to temporarily store excess incoming data. Packet burst sizes, typically 1500 bytes, further influence the required FIFO depth to prevent data loss.

Other factors affecting FIFO depth include latency, burst versus continuous data streams, and synchronous versus asynchronous FIFOs. Latency impacts FIFO depth by determining how much data must be buffered before it is available for processing. Systems with significant propagation delays require deeper FIFOs. Additionally, burst-mode transmissions demand larger FIFOs to accommodate sudden spikes in data traffic, whereas continuous streams may need shallower buffers. In asynchronous FIFOs, depth calculations also depend on clock domain crossing and the need for additional synchronization mechanisms. Verilog implementations of FIFO designs often incorporate status flags such as "full," "empty," "almost full," and "almost empty" to monitor and optimize performance dynamically.

Case – 1 : $f_A > f_B$ with no idle cycles in both write and read.

EXAMPLE 1 :

- Writing frequency = $f_A = 100\text{MHz}$
- Reading Frequency = $f_B = 60\text{MHz}$
- Burst Length = 150 data items
- There are no idle cycles in both reading and writing, meaning all the items in the burst will be written and read in consecutive clock cycles.

SOLUTION:

- Time required to write one data item = $\frac{1}{100\text{ MHz}} = 10\text{ nSec.}$
- Time required to write all the data in the burst = $120 * 12.5\text{ nSec} = 1500\text{ nSec}$
- Time required to read one data item = $\frac{1}{60\text{ MHz}} = 16.67\text{ nSec.}$
- So, for every 16.67 nSec, the module B is going to read one data in the burst.
- In a period of 1500 nSec, 150 data items can be written.
- The number of data items that can be read in 1500 nSec =
$$\frac{1500\text{ nSec}}{16.67\text{ nSec}} = 90$$
- The remaining number of bytes to be stored in the FIFO = $150 - 90 = 60$.
- So, the FIFO in this scenario must be capable of storing 60 data items.
- So, the minimum depth of the FIFO should be 60.

EXAMPLE 2 :

- Writing frequency = $f_A = 90\text{MHz}$
- Reading Frequency = $f_B = 55\text{MHz}$
- Burst Length = 200 data items
- There are no idle cycles in both reading and writing, meaning all the items in the burst will be written and read in consecutive clock cycles.

SOLUTION:

- Time required to write one data item = $\frac{1}{90\text{ MHz}} = 11.11\text{ nSec}$.
- Time required to write all the data in the burst = $200 \times 11.11\text{ nSec} = 2222\text{ nSec}$.
- Time required to read one data item = $\frac{1}{55\text{ MHz}} = 18.18\text{ nSec}$
- So, for every 18.18 nSec, the module B is going to read one data in the burst.
- In a period of 2222 nSec, 200 data items can be written.
- The number of data items that can be read in 2222 nSec =

$$\frac{2222\text{ nSec}}{18.18\text{ nSec}} = 122$$

- The remaining number of bytes to be stored in the FIFO = $200 - 122 = 78$.
- So, the FIFO in this scenario must be capable of storing 78 data items.
- So, the minimum depth of the FIFO should be 78.

Case – 2 : $f_A > f_B$ with one clk cycle delay between two successive reads and writes.

EXAMPLE 1 :

- Writing frequency = $f_A = 80\text{MHz}$
- Reading Frequency = $f_B = 50\text{MHz}$
- Burst Length = 180 data items
- There is a one clock cycle delay between two successive reads and writes.

SOLUTION:

- Time required to write one data item = $\frac{1}{80\text{ MHz}} = 12.5\text{ nSec}$.
- Including one clock cycle delay, the effective write time per item = $12.5\text{ nSec} + 12.5\text{ nSec} = 25\text{ nSec}$
- Time required to write all the data in the burst = $180 \times 25\text{ nSec} = 4500\text{ nSec}$
- Time required to read one data item = $\frac{1}{50\text{ MHz}} = 20\text{ nSec}$.
- Including one clock cycle delay, the effective read time per item = $20\text{ nSec} + 20\text{ nSec} = 40\text{ nSec}$.
- In 4500 nSec , the number of data items that can be written = 180
- The number of data items that can be read in $4500\text{ nSec} = \frac{4500\text{ nSec}}{40\text{ nSec}} = 112$
- The remaining number of bytes to be stored in the FIFO = $180 - 112 = 68$.
- So, the FIFO in this scenario must be capable of storing 68 data items.
- So, the minimum depth of the FIFO should be 68.

EXAMPLE 2 :

- Writing frequency = $f_A = 100\text{MHz}$
- Reading Frequency = $f_B = 60\text{MHz}$
- Burst Length = 150 data items
- There is a one clock cycle delay between two successive reads and writes.

SOLUTION:

- Time required to write one data item = $\frac{1}{100\text{ MHz}} = 10\text{nSec}$.
- Including one clock cycle delay, the effective write time per item = $10\text{ nSec} + 10\text{ nSec} = 20\text{ nSec}$
- Time required to write all the data in the burst = $150 \times 20 = 3000\text{ nSec}$.
- Time required to read one data item = $\frac{1}{60\text{ MHz}} = 16.67\text{nSec}$.
- Including one clock cycle delay, the effective read time per item = $16.67\text{ nSec} + 16.67\text{ nSec} = 33.33\text{ nSec}$
- In 3000 nSec , the number of data items that can be written = 150.
- The number of data items that can be read in $3000\text{ nSec} = \frac{3000\text{ nSec}}{33.33\text{ nSec}} = 90$
- The remaining number of bytes to be stored in the FIFO = $150 - 90 = 60$.
- So, the FIFO in this scenario must be capable of storing 60 data items.
- So, the minimum depth of the FIFO should be 60.

Case — 3 : $f_A > f_B$ with idle cycles in both write and read.

EXAMPLE 1 :

- Writing frequency = $f_A = 90\text{MHz}$
- Reading Frequency = $f_B = 60\text{MHz}$
- Burst Length = No. of data items to be transferred = 150
- No. of idle cycles between two successive writes = 2
- No. of idle cycles between two successive reads = 4

SOLUTION :

- The no. of idle cycles between two successive writes is 2 clock cycles. This means that after writing one data item, module A waits for 2 clock cycles before initiating the next write. So, for every three clock cycles, one data is written.
- The no. of idle cycles between two successive reads is 4 clock cycles. This means that after reading one data item, module B waits for 4 clock cycles before initiating the next read. So, for every five clock cycles, one data is read.
- Time required to write one data item = $3 \times \frac{1}{90 \text{ MHz}} = 33.33 \text{ nSec.}$
- Time required to write all the data in the burst = $150 \times 33.33 \text{ nSec} = 5000 \text{ nSec.}$
- Time required to read one data item = $5 \times \frac{1}{60 \text{ MHz}} = 83.33 \text{ nSec}$
- So, for every 83.33 nSec, module B reads one data item in the burst.

- The number of data items that can be read in 5000 nSec =
$$\frac{5000 \text{ nSec}}{83.33 \text{ nSec}} = 60$$
- The remaining number of bytes to be stored in the FIFO = 150 – 60 = 90
- So, the FIFO in this scenario must be capable of storing 90 data items.
- So, the minimum depth of the FIFO should be 90.

EXAMPLE 2 :

- Writing frequency = $f_A = 80\text{MHz}$.
- Reading Frequency = $f_B = 50\text{MHz}$. Burst Length = No. of data items to be transferred = 120.
- No. of idle cycles between two successive writes is = 1.
- No. of idle cycles between two successive reads is = 3.

SOLUTION:

- The no. of idle cycles between two successive writes is 1 clock cycle. It means that, after writing one data, module A is waiting for one clock cycle, to initiate the next write. So, it can be understood that for every two clock cycles, one data is written.
- The no. of idle cycles between two successive reads is 3 clock cycles. It means that, after reading one data, module B is waiting for 3 clock cycles, to initiate the next read. So, it can be understood that for every four clock cycles, one data is read.

- Time required to write one data item = $2 * \frac{1}{80 \text{ MHz}} = 25 \text{ nSec.}$
- Time required to write all the data in the burst = $120 * 25 \text{ nSec.} = 3000 \text{ nSec.}$
- Time required to read one data item = $4 * \frac{1}{50 \text{ MHz}} = 80 \text{ nSec.}$
- So, for every 80 nSec, the module B is going to read one data in the burst.
- So, in a period of 3000 nSec, 120 no. of data items can be written.
- The no. of data items can be read in a period of 3000 nSec =
$$\left(\frac{3000 \text{ nSec}}{80 \text{ nSec}} \right) = 37.5 = 37 \text{ (approx.)}$$
- The remaining no. of bytes to be stored in the FIFO = $120 - 37 = 83.$
- So, the FIFO which has to be in this scenario must be capable of storing 83 data items.
- So, the minimum depth of the FIFO should be 83.

Case - 4 : $f_A > f_B$ with duty cycles given for wr_enb and rd_enb.

EXAMPLE 1 :

- Writing frequency = $f_A = 100\text{MHz}$
- Reading frequency = $f_B = 60\text{MHz}$
- Burst Length = No. of data items to be transferred = 150
- Duty cycle of wr_enb (write enable) = $40\% = 2/5$
- Duty cycle of rd_enb (read enable) = $30\% = 3/10$

SOLUTION :

- Time required to write one data item = $\frac{1}{100\text{ MHz}} * 5/2 = 25\text{nSec}$
- Time required to write all the data in the burst = $150 \times 25 = 3750\text{ nSec}$
- Time required to read one data item = $\frac{1}{60\text{ MHz}} * 10/3 = 55.56\text{ nSec}$
- The number of data items that can be read in $3750\text{ nSec} = \frac{3750\text{ nSec}}{55.56\text{ nSec}} = 67.5 \approx 67$
- The remaining number of bytes to be stored in the FIFO = $150 - 67 = 83$
- So, the FIFO must be capable of storing at least 83 data items.
- So, the minimum depth of the FIFO should be 83.

EXAMPLE 2 :

- Writing frequency = $f_A = 120\text{MHz}$
- Reading frequency = $f_B = 80\text{MHz}$
- Burst Length = No. of data items to be transferred = 200
- Duty cycle of wr_enb (write enable) = $60\% = 3/5$
- Duty cycle of rd_enb (read enable) = $40\% = 2/5$

SOLUTION:

- Time required to write one data item = $\frac{1}{120\text{ MHz}} * 5/3 = 13.89\text{ nSec.}$
- Time required to write all the data in the burst = $200 \times 13.89 = 2778\text{ nSec}$
- Time required to read one data item = $\frac{1}{80\text{ MHz}} \times \frac{5}{2} = 31.25\text{ nSec}$
- The number of data items that can be read in 2778 nSec =
$$\frac{2778\text{ nSec}}{31.25\text{ nSec}} = 88.9 \approx 89$$
- The remaining number of bytes to be stored in the FIFO = $200 - 89 = 111$
- ✓ So, the FIFO must be capable of storing at least 111 data items.
- So, the minimum depth of the FIFO should be 111.

Case — 5 : $f_A < f_B$ with no idle cycles in both write and read (i.e., the delay between two consecutive writes and reads is one clock cycle).

EXAMPLE 1 :

- Writing frequency = $f_A = 30\text{MHz}$.
- Reading Frequency = $f_B = 50\text{MHz}$.
- Burst Length = No. of data items to be transferred = 120.
- There are no idle cycles in both reading and writing which means that, all the items in the burst will be written and read in consecutive clock cycles.

SOLUTION :

- In this case, a FIFO of depth '1' will be sufficient because, there will not be any data loss since the reading is faster than writing.

EXAMPLE 2 :

- Writing frequency = $f_A = 40\text{MHz}$
- Reading frequency = $f_B = 70\text{MHz}$
- Burst Length = No. of data items to be transferred = 150
- No idle cycles in both reading and writing, meaning all items in the burst will be written and read in consecutive clock cycles.

SOLUTION :

- Since the reading speed (70MHz) is greater than the writing speed (40MHz), data will always be read before the next write happens.
- A FIFO of depth '1' will be sufficient because there will be no data loss, as every data written into FIFO will be read immediately in the next clock cycle.

Case – 6 : $f_A = f_B$ with no idle cycles in both write and read (i.e., the delay between two consecutive writes and reads is one clock cycle).

EXAMPLE 1 :

- Writing frequency = $f_A = f_B = 30\text{MHz}$.
- Burst Length = No. of data items to be transferred = 120.
- There are no idle cycles in both reading and writing which means that, all the items in the burst will be written and read in consecutive clock cycles.

SOLUTION :

- FIFO is not required if there is no phase difference between clkA and clkB .
- A FIFO of depth '1' will be sufficient if there is some phase difference between clkA and clkB .

EXAMPLE 2 :

- Writing frequency = $f_A = 50\text{MHz}$
- Reading frequency = $f_B = 50\text{MHz}$
- Burst Length = No. of data items to be transferred = 100
- No idle cycles in both reading and writing, meaning all items in the burst will be written and read in consecutive clock cycles.

SOLUTION :

- FIFO is not required if there is no phase difference between clkA and clkB .
- A FIFO of depth '1' will be sufficient if there is some phase difference between clkA and clkB .

COMMON MISTAKES & OPTIMIZATION TIPS

1. AVOIDING UNDERFLOW & OVERFLOW

FIFO underflow and overflow are critical issues that can lead to data loss or system failure. Here's how to prevent them:

Solution 1 : Round Up FIFO Depth to the Next Power of 2

- Memory allocation is more efficient when the FIFO depth is a power of 2.
- This simplifies address generation and improves performance.
- Example: Instead of choosing a FIFO depth of 10 or 15, round it up to 16.

Why?

- Most memory architectures are designed for power-of-2 sizes, making address decoding and read/write operations more efficient.
- It reduces hardware complexity when implementing a circular buffer for FIFO.

Solution 2 : Add a 25 – 30% Margin for Unpredictable Variations

- Data flow rates are often not constant due to system variations.
- Having extra FIFO space ensures that unexpected spikes in data rates do not cause overflow.

Example Scenario:

- Suppose your calculations show that a FIFO depth of 100 is needed.
- Instead of allocating exactly 100, allocate 125 to 130 to handle variations safely.

2. CHOOSING AN OPTIMAL FIFO DEPTH

1. If $WR_CLK > RD_CLK$ (Write Clock is Faster than Read Clock)

- Problem: Data accumulates in FIFO faster than it is being read.
- Solution: Increase FIFO depth to store excess data until it can be read.
 - Example: Write frequency = 80 MHz, Read frequency = 50 MHz.
 - Data builds up quickly; a deeper FIFO is needed to avoid overflow.

2. If $RD_CLK > WR_CLK$ (Read Clock is Faster than Write Clock)

- Problem: Data is being read faster than it is written, leading to underflow.
- Solution: A smaller FIFO is sufficient because data is consumed quickly.
 - Example: Write frequency = 30 MHz, Read frequency = 50 MHz.
 - FIFO empties quickly, so a large depth is unnecessary.



Excellence in World class
VLSI Training & Placements

Do follow for updates & enquires



+91- 9182280927